

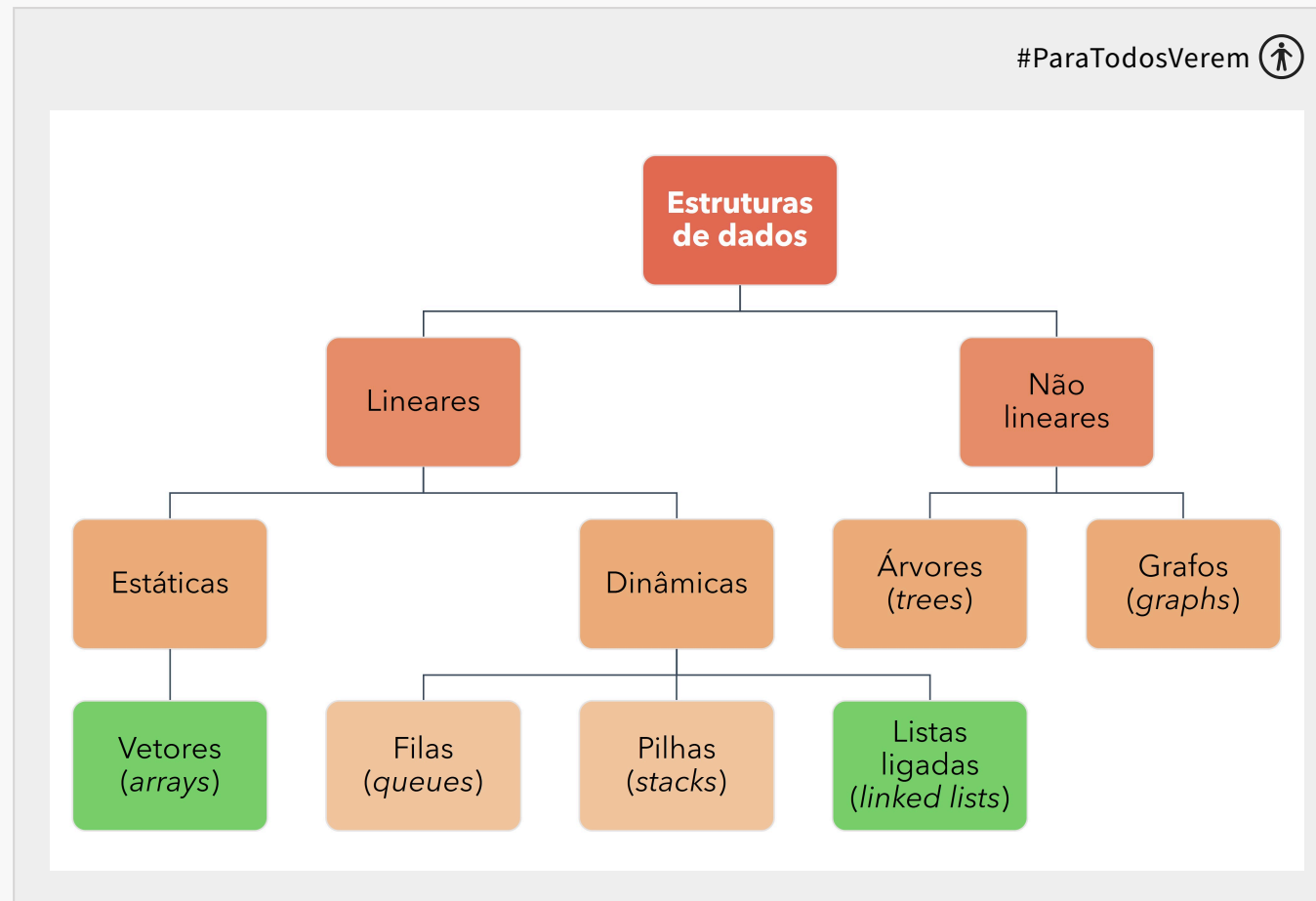


Métodos de Pesquisa e Ordenação em Estruturas de Dados

UNIDADE 02

Estruturas de dados: Listas, vetores e matrizes

Olá! Espero que você esteja bem. A partir desta semana, nos aprofundaremos nos diferentes tipos básicos de estruturas de dados. Por isso, para começar, trabalharemos com dois tipos básicos de estruturas lineares: vetores e listas ligadas. Você verá que existem algumas formas diferentes de trabalharmos com tais estruturas. Vamos lá?



Fonte: O autor (2023).

Praticamente em todos os cursos Tecnologia da Informação (TI), em todas as universidades, aprendemos sobre estruturas de dados. Existe um motivo para isso, e você logo perceberá o quão fundamental é o conhecimento de estruturas como listas, matrizes e vetores. Pense neles como os tijolos fundamentais de uma construção: sem eles, não teríamos como edificar estruturas mais complexas. Assim, antes de mergulharmos

em algoritmos de pesquisa e ordenação mais sofisticados, vamos garantir que compreendemos bem esses componentes essenciais.

Agora, você pode se perguntar: “por que mesmo é tão importante entender essas estruturas? Não seria tão mais simples somente termos um guia rápido de consulta para implementação em qualquer linguagem de programação? Ou um passo a passo para aplicar rapidamente estruturas de dados em diferentes linguagens?”.

Bem, imagine tentar organizar uma biblioteca sem prateleiras, ou tentar entender um problema matemático sem visualizá-lo em um gráfico. Vetores, listas e matrizes nos oferecem maneiras de organizar e representar informações de maneira estruturada. Essas ferramentas tornam possível não apenas armazenar dados, mas manipulá-los de maneira eficaz, possibilitando operações de pesquisa e ordenação eficientes.

Pensando em mercado de trabalho, profissionais bem pagos sabem diferenciar e implementar corretamente diferentes estruturas de dados para cada problema. Infelizmente já tive muitas experiências negativas com sistemas implementados por outras pessoas em que a estrutura de dados incorreta foi usada. Na prática, isso significou em atrasos de projeto, dores de cabeça e estresses que poderiam ser evitados se tais profissionais tivessem passado adequadamente por esta disciplina.



DICA

Os processos seletivos de equipes de TI no Brasil e no exterior que pagam salários acima da média frequentemente requerem conhecimentos práticos em diferentes estruturas de dados.

Por isso, conforme avançamos neste conteúdo, convido você a adotar uma mentalidade de **curiosidade**. Sempre se pergunte: "por que isso funciona dessa maneira?", ou "como essa estrutura pode ser aplicada em situações do mundo real?". Acredite: os melhores profissionais de TI do mundo não são apenas proficientes em manipular essas estruturas, mas também compreendem profundamente por que elas são formadas e funcionam da maneira que são. Vamos juntos neste desafio?

Nesta semana, veremos de forma mais aprofundada as listas estáticas (vetores) e as dinâmicas (listas ligadas).

O que acha de começarmos vendo um exemplo com código? Você poderá ver essa demonstração primeiramente e depois partir para o conteúdo que escrevemos especialmente para você ou, se quiser, pode ver primeiramente o conteúdo escrito e depois retornar a este vídeo.

| Estrutura de dados: Vetor (array)

Ao falarmos de vetores no contexto da programação, muitas vezes nos referimos a uma estrutura que é semelhante às listas, mas com algumas peculiaridades. Um vetor é, tradicionalmente, uma **coleção unidimensional de elementos do mesmo tipo, em que cada elemento tem uma posição única**.



IMPORTANTE

Em Java, o ArrayList já implementa “de fábrica” muitas das coisas que implementaremos manualmente aqui. Como o objetivo é o de entender como esta estrutura funciona na prática, **não** usaremos ArrayLists aqui.

Em muitas linguagens de programação, como Java, os vetores (ou **arrays**) possuem um tamanho fixo, definido no momento da sua criação. Isso significa que, uma vez que um vetor é definido para conter, digamos, 10 elementos, ele sempre terá espaço para 10 elementos, não mais nem menos. Veja o exemplo a seguir.

</>

```
1 public class Main {
2     public static void main(String[] args) {
3         int[] numeros = new int[10];
4
5         for (int i = 0; i < numeros.length; i++) {
6             numeros[i] = i * 5;
7         }
8
9         System.out.println("Conteúdo do array:");
10        for (int num : numeros) {
11            System.out.print(num + " ");
12        }
13    }
14 }
```

Este código define uma classe chamada **main** que possui um método **main()**, que é o ponto de entrada para a execução. Veja que criamos um vetor chamado **numeros** de tamanho 10. Aqui, um **loop for** percorre esse vetor, e para cada posição **i** do vetor, atribui um valor que é igual a $i * 5$.

Depois de preencher o vetor, o programa mostra em um segundo **loop** o seu conteúdo – isto é, os números: **0 5 10 15 20 25 30 35 40 45**.

A simplicidade e a eficiência dos vetores são suas principais vantagens. Acessar um elemento em um vetor, seja para leitura ou escrita, é uma operação extremamente rápida. Estes vetores funcionam como uma série de caixas em uma prateleira, em que cada caixa tem um número. Se você sabe o número da caixa (ou o índice no vetor), pode acessar o conteúdo dessa caixa instantaneamente, sem verificar todas as outras caixas. Por outro lado, essa eficiência vem com o custo da flexibilidade. Como o tamanho do vetor é fixo, se você precisar de mais espaço do que inicialmente alocou, terá que criar um outro vetor e copiar os dados.

Entretanto, na realidade das aplicações modernas, muitas vezes os "vetores" com os quais trabalhamos são abstrações que lidam com essas limitações de tamanho por trás dos panos, como o *ArrayList* em Java. Eles combinam a eficiência do acesso direto aos elementos com a flexibilidade de adicionar ou remover elementos conforme necessário. Ainda assim, é importante entender os princípios fundamentais dos vetores tradicionais, uma vez que eles fornecem a base sobre a qual essas abstrações mais sofisticadas são construídas.

Vetores no mercado de trabalho

Agora, em quais casos de uso reais é preferível o uso de vetores? Ou, sendo mais claro, por que estudar vetores? Vejamos alguns casos de uso:

+ Desenvolvedores de software, em geral

- **Armazenamento e manipulação de dados:** em qualquer aplicação, podemos precisar armazenar conjuntos de itens, seja uma lista de usuários, produtos, mensagens, ou qualquer outro item. Os vetores são frequentemente usados para esse propósito, especialmente quando a **quantidade de itens é conhecida e fixa**.
- **Implementação de algoritmos:** muitos algoritmos, como os de ordenação e pesquisa, usam vetores.

+ Desenvolvedores de jogos

- **Renderização gráfica:** usamos vetores para guardar informações de pixels, texturas, e vetores para renderização gráfica.

- **Mapeamento de cenários:** em jogos 2D ou baseados em *tiles*, podemos usar vetores bidimensionais para representar o mapa ou a grade do jogo.

+ Cientistas e analistas de dados

- **Análise estatística:** é comum usarmos vetores (ou estruturas de dados baseadas em *arrays*, como os *DataFrames* em Python) para armazenar e manipular os dados.
- **Machine learning:** vetores, especialmente vetores multidimensionais (como os tensores), são essenciais para armazenar informações como pesos e *bias* de redes neurais. Se você possui interesse em inteligência artificial é provável que já tenha ouvido falar sobre o TensorFlow: ele é um dos exemplos de implementações que usam vetores.

+ Engenheiros de sistemas e desenvolvedores de firmware

- **Buffer de dados:** em sistemas de baixo nível, os vetores podem ser usados como buffers para armazenar dados temporariamente.
- **Manipulação de portas de entrada e saída:** em desenvolvimento de sistemas embarcados, podemos usar vetores para ler ou escrever sequências de portas ou registradores de microcontroladores.

+ Desenvolvedores web (front-end)

- **Manipulação do DOM:** ao interagir com elementos da página, é comum obter listas ou coleções de elementos. Muitas vezes manipulamos isto como vetores.
- **Animações:** sequências de *frames* ou valores a serem animados são frequentemente armazenados em vetores.

+ Desenvolvedores de aplicativos de celulares e tablets

- **Listas e *scroll views*:** exibir listas de itens, como uma lista de contatos ou mensagens, frequentemente envolve o uso de vetores para armazenar e manipular esses itens.

+ Administradores de sistemas e engenheiros de DevOps

- **Scripting e automação:** ao escrever *scripts* para automação, é comum usarmos vetores para armazenar sequências de comandos, listas de máquinas etc.

+ Desenvolvedores de banco de dados

- **Consulta e manipulação de dados:** enquanto a maioria dos bancos de dados opera em conjuntos de dados, é comum recebermos os resultados em forma de vetores ao conectarmos nestes bancos com algoritmos escritos em linguagens como Java, Python, Scala e C#.



DICA

“array” é outro nome para “vetor”. É comum encontrarmos referências a estes dois nomes na literatura. Portanto, usaremos ambos os termos para nos acostumarmos.

Manipulando vetores

Inicializando vetores

Em Java, vetores (*arrays*) são objetos que armazenam múltiplos valores do mesmo tipo em uma sequência contínua de memória. Vamos usar o código a seguir como exemplo para explicar tais conceitos.

```
</>
```

```
1 public class Main {
2     public static void main(String[] args) {
3         // primeira forma
4         int[] meuArray;
5         meuArray = new int[5];
6
7         meuArray[0] = 1;
8         meuArray[1] = 2;
9         meuArray[2] = 3;
10        meuArray[3] = 4;
11        meuArray[4] = 5;
12
13        // segunda forma
14        int[] outroArray = {1, 2, 3, 4, 5};
15
16        // outros tipos (exemplo: String)
17        String[] nomes = {"Ana", "Beto", "Carlos"};
18    }
19 }
```

Primeira forma: declarando o *array*, e depois o inicializando. Aqui, separamos os passos de declarar um *array* e inicializá-lo (isto é, preencher com alguns valores). Nesta forma, a primeira coisa que você precisa fazer é declarar um *array*. Isso basicamente informa ao Java que você deseja um *array*, mas ainda não especifica o tamanho ou os valores.

```
int[] meuArray;
```

Neste exemplo, estamos informando que gostaríamos de um *array* de números inteiros (int) chamado de **meuArray**. Depois de declararmos, precisamos alocar memória do computador para o *array*. Aqui você especificará quantos elementos o *array* pode conter.

```
meuArray = new int[5];
```

Neste exemplo, queremos que o *array* contenha cinco números inteiros. O segredo está na palavra ***new***: isso significa que estamos inicializando este vetor e, depois disso, poderemos inserir os números. Depois de informarmos quantos elementos o *array* terá, poderemos atribuir os valores para cada índice em seguida:

```
meuArray[0] = 1;  
meuArray[1] = 2;  
meuArray[2] = 3;  
meuArray[3] = 4;  
meuArray[4] = 5;
```

Veja que estamos atribuindo os valores individualmente para cada índice. Aqui, a ordem não importa – podemos inserir os valores para cada índice na ordem que quisermos.

Segunda forma: declarando e inicializando o *array* ao mesmo tempo. Se você já souber quais valores deseja no *array* no momento da declaração, poderá inicializá-lo diretamente como no exemplo a seguir.

```
int[] outroArray = {1, 2, 3, 4, 5};
```

Arrays de outros tipos: da mesma forma que criamos *arrays* de números inteiros também é possível termos *arrays* de outros tipos. Veja o exemplo a seguir, o qual possui um *array* de *strings*.

```
String[] nomes = {"Ana", "Beto", "Carlos"};
```

Adicionando e removendo elementos

Como comentei anteriormente, os *arrays* possuem um tamanho fixo. Uma vez criado, o tamanho do *array* não pode ser alterado. Isso implica que não podemos adicionar ou remover elementos **diretamente**, como faríamos em uma lista ou coleção dinâmica. Por outro lado, existe uma alternativa para isso.

Para adicionar um elemento, você precisará criar um outro *array* com tamanho original + 1. Depois, precisará copiar todos os elementos do *array* original para o novo array. Finalmente, podemos adicionar o novo elemento na posição desejada do novo *array*. Teste o código a seguir como um exemplo.

</>

```
1  import java.util.Arrays;
2
3  public class Main {
4      public static void main(String[] args) {
5          int[] original = {1, 2, 3};
6          System.out.println("Vetor original: " + Arrays.toString(original));
7
8          int[] novo = new int[original.length + 1];
9          System.arraycopy(original, 0, novo, 0, original.length);
10         novo[original.length] = 4000; // Adicionando o elemento 4000
11         System.out.println("Novo vetor: " + Arrays.toString(novo));
12     }
13 }
```

Agora, para remover um elemento, você precisará fazer um processo parecido. Primeiramente, criamos um *array* com tamanho original - 1. Depois, copiamos todos os elementos do *array* original para o novo, exceto o elemento que você deseja remover. Teste o código a seguir como um exemplo.

```
</>

1  import java.util.Arrays;
2
3  public class Main {
4      public static void main(String[] args) {
5          int[] original = {1, 2, 3, 4};
6          int indexParaRemover = 2; // Por exemplo, remover o elemento na posição 2 (ou seja, o número 3)
7
8          int[] novo = new int[original.length - 1];
9
10         // Copiando os elementos que ficam antes do índice a ser removido
11         System.arraycopy(original, 0, novo, 0, indexParaRemover);
12         System.out.println("Novo vetor (parcial): " + Arrays.toString(novo));
13
14         // Copiando elementos que ficam depois do índice a ser removido
15         System.arraycopy(original, indexParaRemover + 1, novo, indexParaRemover, original.length - indexParaRemover);
16         System.out.println("Novo vetor (final): " + Arrays.toString(novo));
17     }
18 }
```

Isso é um processo chato, na verdade. Além disso, ambos os processos são ineficientes em termos de desempenho se comparados a estruturas de dados dinâmicas, como o ***ArrayList*** em Java, que são projetados para adicionar e remover elementos com mais eficiência. Por isso, quando o desempenho e a flexibilidade são uma preocupação, geralmente é aconselhável usar ***ArrayList*** ou outras coleções da Java em vez destes *arrays* simples.

Eu também gostaria de te dar algumas dicas para trabalhar com vetores, uma vez que é um tipo de estrutura de dados que você trabalhará com frequência na sua carreira profissional. Entenda isso como um conselho mesmo, beleza? Vamos usar como uma analogia aqui uma situação na qual você está se preparando para organizar a sua biblioteca pessoal. Neste exemplo, as estantes de livros são como vetores: prateleiras lineares em que você coloca seus livros.

Agora, se você começa a colocar livros aleatoriamente nas prateleiras sem qualquer padrão, da próxima vez que quiser encontrar um livro específico, vai gastar um bom tempo procurando, certo? Por isso, organizar a sua estante de maneira eficiente se torna crucial. Vamos aplicar essa analogia ao mundo dos vetores e entender algumas das melhores práticas para lidar com eles.

+ **Pense antes de redimensionar**

Assim como não é simples trocar uma estante antiga por uma nova estante (e maior), expandir um vetor requer tempo e esforço. Em linguagens de programação, isso geralmente envolve criar um vetor com um maior tamanho e copiar todos os elementos do vetor antigo. Portanto, sempre que possível, tente estimar o tamanho de que precisa com antecedência para evitar redimensionamentos frequentes.

+ **Use a estrutura certa**

Se você souber que irá adicionar e remover livros (ou elementos) com frequência, talvez uma prateleira tradicional (ou vetor) não seja a melhor escolha. Existem outros tipos de "prateleiras" mais flexíveis, como as ***ArrayLists*** em Java,

que facilitam o processo de adição e remoção, sem a necessidade de "construir uma nova prateleira" toda vez.

+ Mantenha-o ordenado quando necessário

Se você mantém seus livros em ordem alfabética, é mais fácil encontrar um quando precisa. Da mesma forma, se você mantiver um vetor ordenado, operações como pesquisa (especialmente pesquisa binária) serão muito mais rápidas. Veremos um pouco mais sobre isso nesta disciplina, mais adiante. Dito isso, lembre-se: ordenar tem um custo. Se você estiver constantemente recebendo novos livros e os reordenando sempre, isso pode não ser eficiente. O segredo é encontrar um equilíbrio.

+ Acesse com cuidado

Imagine tentar pegar um livro da décima prateleira de uma estante que só tem cinco. Isso seria impossível, certo? Da mesma forma, tentar acessar ou modificar um índice que não existe em um vetor resultará em um erro. Portanto, sempre verifique quantos elementos o seu vetor existe antes de acessar ou modificar seus elementos.

A arte de buscar informações em vetores é uma das habilidades fundamentais que todo programador deve dominar. Afinal, por que usaríamos um vetor, ou qualquer estrutura de dados, se não pudermos recuperar as informações armazenadas nele de maneira eficiente? Existem vários métodos clássicos de pesquisa em vetores, cada um com suas vantagens, desvantagens e casos de uso ideais.

Para aprender sobre estes métodos será importante colocarmos a mão na massa e os implementarmos do zero. Isso é importante, uma vez que existe uma grande diferença entre ler sobre alguma coisa na teoria, e ser capaz de implementá-lo na prática. Implementar um algoritmo manualmente ajuda a solidificar o seu entendimento sobre o conceito.



IMPORTANTE

No mercado de trabalho será muito raro ter que implementar estes métodos de pesquisa e ordenação manualmente, do zero. Por outro lado, saber os seus prós e contras depois de ter uma experiência prática fará toda a diferença. Afinal, pensemos: para o mercado, qual é a diferença entre alguém que decorou conceitos por três anos e um outro profissional que implementou aqueles conceitos na prática por três anos dentro de uma empresa? A resposta é a mesma: a experiência prática.

Pesquisa linear

Começemos com a **pesquisa linear**, o método mais simples e direto. Imagine que você tem uma linha de livros e precisa encontrar um título específico. Você começaria do início e examinaria cada livro até encontrar o que procura. Da mesma forma, na pesquisa linear, percorremos o vetor elemento por elemento até encontrarmos o que buscamos ou confirmarmos que o elemento não está presente. Apesar de ser fácil de implementar, sua eficiência pode ser um problema em vetores muito grandes, pois, no pior caso, pode exigir a verificação de cada elemento. Teste o exemplo a seguir (de verdade – teste! É melhor do que somente bater o olho no código e avançar para o conteúdo seguinte):

```
</>
```



```
1 public class Main {
2     public static int buscaLinear(int[] array, int x) {
3         for (int i = 0; i < array.length; i++) {
4             System.out.println("Procurando... Tentativa " + (i + 1) + " de " + array.length);
5             if (array[i] == x) {
6                 return i; // elemento encontrado na posição 'i'
7             }
8         }
9         return -1; // elemento não encontrado
10    }
11
12    public static void main(String[] args) {
13        int[] vetor = {4, 2, 7, 9, 11, 15, 5, 8, 3, 6};
14        int item_para_encontrar = 5;
15
16        int resultado = buscaLinear(vetor, item_para_encontrar);
17
18        if (resultado == -1) {
19            System.out.println("Elemento não encontrado.");
20        }
21    }
22 }
```

Veja no exemplo apresentado que estamos procurando a posição do valor 5 dentro de um elemento com 10 valores. Se você testou o código verá que tivemos que testar 7 vezes (isto é, percorrer aquele *loop* dentro do método **buscaLinear()** 7 vezes) até encontrar o valor – afinal de contas, ele é o sétimo elemento da lista (índice 6).



IMPORTANTE

Sempre começamos a contar os elementos a partir do zero. Portanto, o primeiro elemento ocupa o índice 0, o segundo elemento ocupa o índice 1, o terceiro elemento ocupa o índice 2, e assim sucessivamente. Por isso, o sétimo elemento ocupa o índice 6.

A Implementação dessa busca é simples, mas o problema pode estar na escalabilidade: imagine um vetor com um milhão de elementos. Se o nosso valor sendo pesquisado está no início do vetor, encontraremos aquele valor com uma relativa velocidade. Agora, se está no final do vetor, teremos que testar quase um milhão de vezes até encontrar o valor. Isto é: encontrar o valor que queremos pode ser um processo muito rápido ou muito lento – tudo depende da posição do valor que estamos pesquisando dentro do vetor e o tamanho do vetor em si.

Pesquisa binária

Contrastando com a pesquisa linear, temos a **pesquisa binária**. É um método mais refinado que se aplica a vetores que foram ordenados com antecedência. Voltando à analogia dos livros, imagine que os livros estejam organizados em ordem alfabética. Em vez de começar do início, você poderia abrir no meio e, dependendo se o livro que procura vem antes ou depois na ordem alfabética, você pode ignorar metade da linha de livros. Repetindo esse processo de dividir pela metade, você encontrará rapidamente o livro desejado ou determinará que ele não está lá. A pesquisa binária é extremamente eficiente em comparação com a pesquisa linear, especialmente em vetores grandes, mas **requer que o vetor esteja ordenado**. Teste o exemplo a seguir.

</>

```
1 public class Main {
2     public static int buscaBinaria(int[] array, int x) {
3         int esquerda = 0;
4         int direita = array.length - 1;
5         int contador = 1;
6
7         while (esquerda <= direita) {
8             int meio = (esquerda + direita) / 2;
9
10            System.out.print("Tentativa " + contador + ". ");
11            System.out.print("Vendo se " + x + " fica antes/depois/é igual a " + array[meio] + ". ");
12            contador++;
13
14            if (array[meio] == x) {
15                System.out.println("Achei!");
16                return meio; // elemento encontrado na posição 'meio'
17            } else if (array[meio] < x) {
18                System.out.println("Fica depois.");
19                esquerda = meio + 1; // procurar na metade à direita
```

Veja que este algoritmo tentou encontrar um valor (11) dentro de um vetor que já estava ordenado. Ele começou pela metade do vetor (13). Como 11 fica antes de 13, então ele tentou procurar novamente, mas somente na primeira metade do vetor. Nesta primeira metade, ele novamente dividiu em duas partes (em que o valor que fica no meio era o 7). Como 11 fica depois de 7, então ele entendeu que o valor a ser pesquisado ficou mais à direita. Ele faz mais duas comparações até encontrar o resultado. Em outras palavras, se esta busca demorou 4 tentativas, na busca linear demoraria 7 tentativas. Portanto, é mais eficiente.



IMPORTANTE

Algoritmos de pesquisa e algoritmos de busca tratam da mesma coisa. Portanto, às vezes podemos nos referir a um

algoritmo como “busca linear” e, em outros casos, como “pesquisa linear”.

Além desses dois métodos, existem várias outras técnicas e algoritmos avançados como, por exemplo, a **pesquisa por interpolação** e a **pesquisa hash**, adaptadas para cenários específicos e otimizadas para diferentes configurações de dados. A busca por interpolação é uma variação da busca binária, utilizada principalmente em listas ordenadas. Em vez de escolher o ponto médio (como na busca binária), a busca por interpolação estima a posição do elemento desejado com base nos valores das extremidades da lista. Já a busca *hash* envolve a utilização de uma estrutura de dados chamada tabela *hash*. Essa estrutura permite o acesso a elementos em tempo constante médio, tornando-a extremamente eficiente para operações de busca.

Independentemente do método, a essência da pesquisa permanece a mesma: acessar e recuperar informações de um conjunto de dados de maneira eficiente. Conforme avançamos em nossos estudos, é essencial não apenas conhecer os algoritmos, mas também entender suas características para escolher o método mais adequado para cada situação.

Resumo

1. Vetores são sequências fixas de elementos: precisamos informar com antecedência *quantos* elementos aquele vetor terá.
2. São sequências unidimensionais de elementos do mesmo tipo, em que cada elemento tem uma posição única.
3. Não dá para adicionar e remover elementos: no máximo, teríamos que criar um segundo vetor com os ajustes.
 - Apesar disso, o Java possui um jeito fácil para manipularmos vetores com o ***ArrayList***.

| Estrutura de dados: Matriz

As matrizes são uma variação dos vetores. Se consideramos os vetores como uma série de caixas em uma única prateleira, então podemos pensar nas matrizes como várias dessas prateleiras empilhadas verticalmente, formando um armário. Uma matriz é basicamente uma coleção bidimensional, frequentemente visualizada **como uma tabela com linhas e colunas**.

Cada elemento na matriz é identificado por **dois índices**, em vez de um: o índice da linha e o índice da coluna. E assim como os vetores, as matrizes tradicionais em muitas linguagens de programação têm tamanho fixo. Ou seja, uma vez que você define uma matriz de 5x5, por exemplo, ela terá sempre cinco linhas e cinco colunas.

No contexto de programação e algoritmos, as matrizes desempenham um papel crucial em muitas aplicações. Usamos matrizes para representar gráficos e redes, para realizar operações de álgebra linear em gráficos computacionais, ou simplesmente para armazenar dados em formatos tabulares. Independentemente do caso, as matrizes são uma ferramenta indispensável no arsenal de um desenvolvedor de *software*. Afinal de contas, por ter duas dimensões, temos aqui uma maneira intuitiva de organizar e manipular dados: especialmente em situações em que a relação entre os dados possui uma natureza bidimensional ou até mesmo multidimensional.



DICA

Imagens podem ser representadas como matrizes. Cada pixel é uma combinação de linha e coluna, e representamos as cores como um número. Este tipo de manipulação de imagens é útil em aplicações como computação gráfica, inteligência artificial, e desenvolvimento de jogos.

No entanto, assim como acontece com os vetores, é importante sabermos das implicações de desempenho ao trabalhar com matrizes. De forma geral, acessar um elemento em uma matriz é um processo rápido, mas operações como a multiplicação de matrizes podem ser computacionalmente intensivas dependendo do tamanho da matriz.

Manipulando matrizes

Inicializando matrizes

Vou adaptar aquele exemplo de inicialização de *arrays* para que funciona com matrizes. Veja o código a seguir como um exemplo.

```
</>
```

```
1 public class Main {
2     public static void main(String[] args) {
3         // primeira forma
4         int[][] minhaMatriz;
5         minhaMatriz = new int[2][3];
6
7         minhaMatriz[0][0] = 1;
8         minhaMatriz[0][1] = 2;
9         minhaMatriz[0][2] = 3;
10        minhaMatriz[1][0] = 4;
11        minhaMatriz[1][1] = 5;
12        minhaMatriz[1][2] = 6;
13
14        // segunda forma
15        int[][] outraMatriz = {
16            {1, 2, 3},
17            {4, 5, 6}
18        };
19    }
```

Primeira forma: declarando o *array*, e depois o inicializando. Veja que a sintaxe é muito parecida com os *arrays*:

```
int[][] minhaMatriz;
```

Neste exemplo, estamos informando que gostaríamos de uma matriz de números inteiros (int) chamada de **minhaMatriz**. Veja que existem dois conjuntos de colchetes depois do tipo de dados (isto é, *int[][]*). O primeiro conjunto de colchetes representam as linhas, e o segundo conjunto representa as colunas. Depois de declararmos, precisamos alocar memória do computador para a matriz. Aqui você especificará quantas linhas e quantas colunas a matriz terá.

```
minhaMatriz = new int[2][3];
```

Neste exemplo, queremos que o array contenha duas linhas e três colunas. Depois de informarmos quantas linhas e quantas colunas a matriz terá, poderemos atribuir os valores para cada índice em seguida:

```
minhaMatriz[0][0] = 1;  
minhaMatriz[0][1] = 2;  
minhaMatriz[0][2] = 3;  
minhaMatriz[1][0] = 4;  
minhaMatriz[1][1] = 5;  
minhaMatriz[1][2] = 6;
```

Veja que estamos atribuindo os valores individualmente para cada índice. *minhaMatriz[0][1]* equivale à primeira linha (índice 0) e à segunda coluna (índice 1). Já *minhaMatriz[1][0]* equivale à segunda linha (índice 1) e à primeira coluna (índice 0), por exemplo. Portanto, veja que primeiro sempre informamos o índice referente à linha e, depois, à coluna.

Segunda forma: declarando e inicializando a matriz ao mesmo tempo. Se você já souber quais valores deseja na matriz no momento da declaração, poderá inicializá-lo diretamente como no exemplo a seguir.

```
int[][] outraMatriz = {  
    {1, 2, 3},  
    {4, 5, 6}  
};
```

Neste caso, {1, 2, 3} equivale ao conteúdo da primeira linha, e {4, 5, 6} equivale ao conteúdo da segunda linha. Poderíamos cadastrar uma terceira linha com outros valores, como {500, 600, 0}, e assim sucessivamente.

Matrizes de outros tipos: da mesma forma que criamos matrizes de números inteiros também é possível termos matrizes de outros tipos (este texto é praticamente igual à explicação de vetores, para você ter uma ideia). Veja o exemplo a seguir, o qual possui uma matriz de *strings* – perceba que seguimos o mesmo padrão de determinar a matriz com “{}” e, dentro deste conjunto de chaves, criamos cada linha com um novo conjunto de “{}”. Aqui, `{"Carlos", "Diana"}` é a segunda linha.

```
String[][] nomes = {  
    {"Ana", "Beto"},  
    {"Carlos", "Diana"},  
    {"Eduardo", "Fernanda"}  
};
```

Adicionando e removendo elementos:

Se os vetores possuem tamanho fixo, isso se aplica a uma matriz. Por isso, precisaremos criar uma outra matriz toda vez que for preciso adicionar uma linha ou coluna. Teste o código a seguir como um exemplo.

```
</>
```



```
1 import java.util.Arrays;
2
3 public class Main {
4     public static void main(String[] args) {
5         int[][] matrizOriginal = {
6             {1, 2, 3},
7             {4, 5, 6}
8         };
9         System.out.println("Matriz original: " + Arrays.deepToString(matrizOriginal));
10
11         // Adicionando uma nova linha
12         int[][] matrizComLinhaAdicional = new int[matrizOriginal.length + 1][matrizOriginal[0].length];
13         for (int i = 0; i < matrizOriginal.length; i++) {
14             for (int j = 0; j < matrizOriginal[i].length; j++) {
15                 matrizComLinhaAdicional[i][j] = matrizOriginal[i][j];
16             }
17         }
18         // Inicializando a nova linha com valores (por exemplo, 0)
19         for (int j = 0; j < matrizComLinhaAdicional[0].length; j++) {
```

Veja que este código possui dois exemplos diferentes: um para criar uma linha e, outro, para criar uma coluna. Em ambos os casos, copiamos todo o conteúdo da matriz original e, depois, criamos mais uma linha ou coluna. Veja que usamos o *Arrays.deepToString()* também: este método do Java permite mostrar o conteúdo de uma matriz na tela.

Agora, para remover um elemento, você precisará fazer um processo parecido. Primeiramente, criamos uma matriz com tamanho original – 1 (seja menos uma linha ou menos uma coluna). Depois, copiamos todos os elementos da matriz original para a nova matriz, exceto o elemento que você deseja remover. Teste o código a seguir como um exemplo de remoção de uma linha.

</>

```
1  import java.util.Arrays;
2
3  public class Main {
4      public static void main(String[] args) {
5          int[][] original = {
6              {1, 2, 3},
7              {4, 5, 6},
8              {7, 8, 9},
9              {10, 11, 12}
10         };
11
12         int linhaParaRemover = 2; // Remover a terceira linha (índice 2)
13
14         int[][] novaMatriz = new int[original.length - 1][original[0].length];
15
16         // Copiando as linhas que ficam antes da linha a ser removida
17         for (int i = 0; i < linhaParaRemover; i++) {
18             System.arraycopy(original[i], 0, novaMatriz[i], 0, original[i].length);
19         }
```

Resumo

1. Matrizes são vetores multidimensionais: geralmente, com duas dimensões que representam linhas e colunas.
2. As mesmas regras sobre vetores também se aplicam às matrizes.

| Estrutura de dados: Listas ligadas

Se você não tinha experiência prévia em programação antes de ingressar no curso, a lista foi a primeira estrutura de dados que você aprendeu na disciplina Raciocínio Computacional, com Python. É uma ferramenta tão aparentemente simples, mas com uma profundidade surpreendente. Você já deve se lembrar de que, em sua essência, uma lista é uma coleção ordenada de elementos, em que cada item tem uma posição definida. **Mas, ao contrário dos *arrays* em que o tamanho é fixo, as listas são dinâmicas em muitas linguagens de programação como Java e Python.** Isso permite que os elementos sejam adicionados ou removidos com facilidade.

A vantagem e beleza das listas está em sua flexibilidade. Jamais deixando de lado as analogias que usamos até o momento, vamos supor que você está planejando uma festa e criou uma lista de convidados. Se no último minuto um amigo que estava viajando decide participar, você pode simplesmente adicionar seu nome sem recriar toda a lista com uma lista dinâmica. E é essa **adaptabilidade** das listas que as torna tão valiosas em programação, especialmente quando não sabemos o tamanho da nossa coleção de dados de antemão.

No entanto, com grande poder vem grande responsabilidade (juro que tentei pensar em uma frase melhor). As listas oferecem muitas funcionalidades práticas, mas é essencial entender o custo associado a cada operação. Adicionar ou remover elementos, especialmente em certas posições, pode ter implicações de desempenho que podem ser significativas dependendo da aplicação. Então, enquanto exploramos mais profundamente as listas, eu te convido a pensar não apenas em “como” usar, mas também nas implicações “por trás das cortinas” de cada ação.

Listas e listas encadeadas/ligadas

Bem, “lista” é uma palavra tão comum que pode facilmente nos confundir. No mundo da ciência da computação, quando falamos de “listas”, podemos estar nos referindo a dois conceitos bem diferentes: **listas** (e aqui incluímos e **listas encadeadas (ou listas ligadas)**). Ambas possuem o propósito de armazenar uma sequência de elementos, mas a forma como fazem isso é bem diferente. E é essa diferença que torna uma mais adequada para certas situações do que a outra.



IMPORTANTE

A diferença entre “vetor” e “lista” é muito confusa porque nem todas as linguagens de programação as usam com o mesmo nome e do mesmo jeito. Nesta disciplina usamos a mesma convenção da literatura: **listas de tamanho fixo == vetores**. Por outro lado, **listas de tamanho dinâmico = listas**. Usaremos este conceito por toda a disciplina. Por exemplo,

o JavaScript possui somente listas, e as chama de arrays (confuso, eu sei). Já a linguagem C e Assembly possuem somente vetores por padrão, e não possuem listas. Finalmente, linguagens como C++, Java, Rust e Swift possuem ambas.

Os *arrays* funcionam como se fossem prateleiras em uma biblioteca. Eles possuem um tamanho definido e cada "prateleira" (ou posição) pode armazenar um "livro" (ou item). Se você quiser adicionar ou remover um livro bem no meio da prateleira, terá que mover todos os outros livros para fazer espaço ou fechar a lacuna. Isso pode ser trabalhoso, certo? Por outro lado, as listas encadeadas funcionam como se fossem uma caça ao tesouro. Cada dica (ou elemento dessa lista) leva você à próxima, e cada dica possui uma parte do tesouro (o valor do elemento) e a localização da próxima dica (qual é o próximo elemento). Se quiser adicionar ou remover uma dica no meio do caminho, basta atualizar a localização dessas dicas: não é necessário mover todo o tesouro.

Listas encadeadas no mercado de trabalho

Então, qual é a melhor estratégia para você usar: prateleiras, ou caça ao tesouro? Às vezes, queremos a eficiência de acessar qualquer livro diretamente de sua prateleira (como em *arrays*). Outras vezes, preferimos a flexibilidade de mudar a ordem das dicas sem afetar o restante da caça ao tesouro (como em listas ligadas, que também chamamos por listas encadeadas).

Agora, em quais casos de uso reais é preferível o uso de listas? Vamos dar uma olhada em alguns dos casos mais comuns de uso de listas encadeadas no mercado de trabalho e os perfis de TI que possam usar essas estruturas:

+ Desenvolvimento de software, em geral

- **Aplicações de memória eficiente:** listas encadeadas são úteis quando o tamanho exato da lista não é conhecido antecipadamente e ela pode mudar dinamicamente. Elas não desperdiçam memória como os *arrays*, uma vez que não é necessário pré-alocar espaço em memória para usá-las.

- **Implementação de estruturas de dados avançadas:** estruturas como pilhas e filas podem ser facilmente implementadas usando listas encadeadas. A propósito: aprenderemos sobre pilhas e filas em breve.
- **Desenvolvedores de sistemas operacionais:** usamos listas encadeadas para gerenciar recursos como, por exemplo, processos que estejam atualmente em execução ou na fila para execução.

+ Engenheiros de softwares de alto desempenho

- **Aplicações que requerem inserções e remoções rápidas, como *big data*:** ao contrário dos *arrays*, as inserções e remoções em listas encadeadas são operações $O(1)$ quando sabemos a posição na qual queremos inserir ou remover dados.

+ Desenvolvedores de jogos

- **Gerenciamento de entidades:** as listas encadeadas podem ser usadas para gerenciar entidades que entram e saem do jogo dinamicamente. Exemplos incluem itens e NPCs que são rapidamente adicionados e removidos de cena.

+ Desenvolvedores de aplicações em tempo real

- **Gerenciamento de tarefas:** existem sistemas de tempo real em que a sequência de tarefas é variável e dinâmica. Aqui, as listas encadeadas podem ser usadas para gerenciar essas tarefas.

+ Desenvolvedores de bancos de dados

- **Gerenciamento de blocos de memória:** os bancos de dados muitas vezes usam listas encadeadas para gerenciar blocos de memória. Em especial, em situações em que registros são frequentemente inseridos e removidos.

+ Desenvolvedores de aplicativos de celulares e *tablets*

- **Uso eficiente de memória:** celulares e *tablets* possuem pouca memória dependendo do modelo. Por isso, usamos listas encadeadas em aplicativos em que temos muitos dados e o uso de memória deve ser otimizado.

Listas encadeadas simples

As **listas encadeadas simples** permitem que deixemos de armazenar os dados em locais contíguos de memória (como ocorre com os vetores) e passemos a guardar os dados em diferentes espaços da memória. Isso é importante porque, na prática, pense em uma memória de computador como uma sala de cinema quase cheia: você precisa encontrar alguns lugares disponíveis, mas não é sempre uma garantia de que todos os lugares livres estejam um ao lado do outro.

Para que isso seja possível, as **listas encadeadas simples possuem duas informações**: cada elemento (também chamado de nó) armazena o dado em si e uma referência (ou ponteiro) para o próximo elemento da lista. É a referência que confere à lista encadeada uma grande flexibilidade. Por exemplo: inserir ou remover um elemento no meio da lista não requer o deslocamento de todos os outros elementos, como seria o caso em um vetor. No entanto, essa vantagem vem com uma desvantagem: o acesso direto a um elemento pelo índice não é constante, pois é necessário percorrer a lista desde o início até chegar ao elemento desejado. A figura a seguir demonstra uma lista, em que cada elemento (nó) possui uma informação e um apontamento para o próximo elemento (lista encadeada).

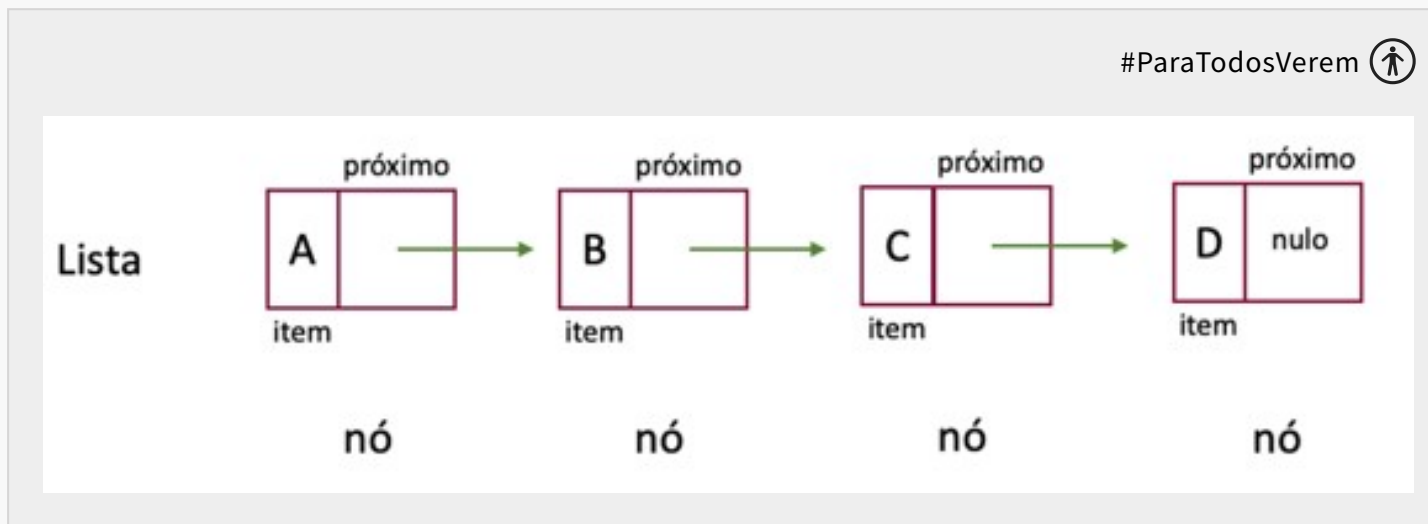


Figura 2: Estrutura de dados, lista. Fonte: O autor (2023).

Para que isso fique mais claro, o que acha de criarmos uma lista encadeada do zero?



IMPORTANTE

O Java já possui listas encadeadas prontas “de fábrica”. O nome é **LinkedList**. Por outro lado, como nem toda linguagem possui isto, é importante que saibamos como isso funciona. Por isso, aprenderemos a criar a nossa própria implementação do zero.

</>

```
1  class Node {
2      int data;
3      Node next;
4
5      public Node(int data) {
6          this.data = data;
7          this.next = null;
8      }
9  }
10
11 class ListaEncadeada {
12     Node head;
13
14     public void append(int data) {
15         Node newNode = new Node(data);
16         if (head == null) {
17             head = newNode;
18             return;
19         }
```

Vamos entender melhor este código com calma? Vejamos algumas coisas:

1. Classe **node**: se um vetor de inteiros armazenava somente números inteiros e um vetor de *strings* armazenava somente *strings*, a nossa lista encadeada sempre armazena nós (**nodes**, em inglês), independentemente do tipo. Cada nó possui duas informações, lembra? Neste caso, temos o dado (variável **data**, que neste exemplo estou usando números inteiros), e uma referência ao próximo nó (variável **next**).
2. Classe **main**: foi criada para demonstrarmos o funcionamento de uma lista encadeada. Veja que no método **main()** criamos uma lista encadeada com três números. Em seguida, mostramos o conteúdo desta lista na tela. Finalmente, removemos um dos elementos que está no meio da lista e mostramos o resultado.

Adicionando e removendo elementos

Agora, como funciona para adicionar e remover elementos dessa lista encadeada? Implementamos isso dentro da classe **ListaEncadeada**. Observe que precisamos saber somente qual é o primeiro nó da lista (variável **head**). Afinal, dentro deste primeiro nó saberemos qual é o próximo nó, e assim sucessivamente. Também saberemos que a lista acabou quando chegarmos a um nó sem referências (ou seja, quando o **next** for igual a *null*). Vejamos os métodos que implementamos:

1. Método **append**: temos aqui uma implementação para incluirmos novos dados nesta lista.
 - Criamos um nó a partir dos dados que recebemos. Veja que este novo nó não possui referências a um próximo nó (uma vez que é o último elemento que será adicionado na lista).
 - Se o primeiro nó (*head*) estiver nulo, então esta lista está vazia. Neste caso, só substituímos o *head* com o nó que acabamos de criar.
 - Caso contrário, precisaremos atualizar as referências da nossa lista. Neste caso, percorreremos todos os nós da lista encadeada até chegar no último nó (isto é, o nó sem referências a um outro nó). Ao chegar neste nó, atualizaremos as referências para que o último nó da lista aponte para o nó que acabamos de criar.
2. Método **printList**: é um método para mostrar esta noção de listas encadeadas. Veja que sempre um elemento apontará para o próximo até chegar ao último elemento, que aponta para lugar algum (*null*).
3. Método **remove**: é um método de remoção de elementos. A remoção de nós em uma lista encadeada pode ser um pouco mais desafiadora do que a adição porque precisamos levar em consideração diferentes cenários. Por exemplo, a remoção pode ser no começo (*head*) da lista, no meio ou no final. Assim que encontramos o nó a ser removido é simples: atualizamos a referência (o parâmetro **next**) que antes se referia a este nó para que aponte ao nó subsequente. Algo como:
 - 10 aponta para 20, 20 aponta para 30, 30 aponta para *null* (uma vez que está no fim da lista).
 - Neste exemplo, queremos remover o 20:

1. Por isso, o 10 não pode mais apontar para o 20.
2. O 10 precisa apontar para o elemento que vem depois do 20: neste caso, é o 30.
3. Por isso, o 10 agora aponta para 30, e 30 continua apontando para *null* (uma vez que está no fim da lista).

Pesquisa

Em listas encadeadas simples, a pesquisa geralmente é **linear**: ela começa no nó “*head*” e prossegue sequencialmente por meio de cada nó, seguindo os ponteiros (ou conexões) até que o item desejado seja encontrado ou até que o final da lista seja alcançado. Veja só: como no pior cenário você terá que verificar todos os nós da lista, temos, portanto, uma complexidade de tempo de $O(n)$ para a pesquisa em listas encadeadas.

O motivo de fazermos isto é que, ao contrário de *arrays*, em que você pode saltar diretamente para um índice específico, em listas encadeadas você precisa percorrer os nós um por um. A desvantagem óbvia deste método é que pode ser um processo muito lento: principalmente se o item que você está procurando estiver no final da lista ou não estiver na lista.

Listas duplamente encadeadas

Outro tipo de lista encadeada é a **lista duplamente encadeada**. Este tipo de lista leva a ideia das listas encadeadas simples um passo adiante. Em vez de cada nó ter uma única referência para o próximo elemento, **ele tem duas referências: uma para o próximo elemento e outra para o elemento anterior**. Esta estrutura bidirecional permite que a lista seja percorrida em ambas as direções, do início ao fim e vice-versa. Isso torna operações como inserção e remoção em ambas as extremidades da lista mais eficientes. No entanto, essa eficiência vem ao custo de armazenamento adicional para o ponteiro extra em cada nó e uma complexidade adicional na manutenção de dois ponteiros. Em resumo: é mais rápido, mas usa mais espaço em memória.

Um dos benefícios deste tipo de lista é a facilidade com que podemos navegar para frente e para trás tornando-a ideal para certas aplicações, como aqueles botões de voltar e avançar em navegadores de páginas da *web* ou os botões de avançar e voltar músicas e vídeos em *playlists* de aplicativos como o Spotify e o YouTube. No entanto, uma lista encadeada simples pode ser preferível devido à sua estrutura mais simples e ao uso reduzido de memória em aplicações em que essa navegação bidirecional não é uma necessidade.

Agora, como criamos a nossa lista duplamente encadeada do zero? Teste o exemplo a seguir, feito em Java.

</>

```
1  class Node {
2      int data;
3      Node anterior;
4      Node proximo;
5
6      public Node(int data) {
7          this.data = data;
8          this.anterior = null;
9          this.proximo = null;
10     }
11 }
12
13 class ListaDuplamenteEncadeada {
14     Node head;
15     Node tail;
16
17     // Adicionar ao final da lista
18     public void append(int data) {
19         Node newNode = new Node(data);
```

Adicionando e removendo elementos

Este código foi adaptado do nosso exemplo da lista encadeada simples. Vamos entendê-lo melhor? Vejamos algumas mudanças:

1. Classe **node**: se antes guardávamos somente o próximo nó (atributo **proximo**, sem acentos), agora armazenamos o nó antecedente também (atributo **anterior**).
2. A classe **ListaEncadeada** agora se chama **ListaDuplamenteEncadeada**.
 - Além de guardarmos o primeiro nó (**head**, ou cabeça), agora armazenamos também o último (**tail**, ou cauda).

- No método ***append***:
 1. Se o primeiro nó (*head*) estiver nulo, então esta lista está vazia. Neste caso, só substituímos o *head* e o *tail* com o nó que acabamos de criar (ou seja: o começo e o fim da lista são iguais).
 2. Caso contrário, precisaremos atualizar as referências da nossa lista. Este processo é mais rápido em relação à lista encadeada simples. Se antes percorríamos todos os nós da lista encadeada até chegar no último nó, agora vamos diretamente ao último nó (*tail*) e atualizamos a sua referência de próximo nó para o nó que estamos adicionando. Da mesma forma, também informamos dentro do nosso novo nó de que o seu anterior é o *tail*. Para finalizar, o *tail* passa a ser este novo nó. **Como uma analogia, é como se você chegasse a uma fila de um caixa de supermercado, avisasse à última pessoa de que você agora está atrás dela e que ela está na nossa frente e, por fim, anunciasse que você passou a ser a última pessoa da fila.**
- Método ***printList***: é um método para mostrar esta noção de listas encadeadas. Veja que sempre um elemento apontará para o próximo até chegar ao último elemento, que aponta para lugar algum (*null*).
- Método ***remove***: também atualizamos a lógica, e ela passa a ser um pouco mais complexa. Ao encontrarmos um nó, precisamos atualizar duas referências. Para que isso fique mais claro, imagine três pessoas em uma fila na seguinte ordem: Alice -> Beatriz -> Carla.
 1. Se a Alice (*head*) sai da fila (Alice -> Carla), informamos que Beatriz é a nova *head*, e que não tem ninguém na frente dela.
 2. Se a Carla (*tail*) sai da fila (Alice -> Beatriz), informamos que Beatriz é a nova *tail*, e que não tem mais ninguém atrás dela.
 3. Se a Beatriz sai da fila (Alice -> Carla), informamos que a próxima pessoa depois da Alice passa a ser Carla. Em seguida, informamos que a pessoa que está na frente da Carla é a Alice.

Pesquisa

Embora listas duplamente encadeadas ofereçam a capacidade de se mover tanto para a frente quanto para trás na lista, o método de pesquisa permanece fundamentalmente semelhante ao das listas encadeadas simples. Ainda é uma pesquisa sequencial, começando do nó de cabeça e movendo-se para o próximo nó até encontrar o item desejado. No entanto, em certas circunstâncias, saber que você está mais próximo do final da lista pode permitir que você comece a pesquisa a partir do final (nó da cauda) e vá para trás. Por outro lado, a pesquisa nas listas duplamente encadeadas leva aproximadamente o mesmo tempo que nas listas encadeadas simples. Isso acontece porque a principal vantagem das listas duplamente encadeadas é a **inserção e remoção eficientes em ambas as extremidades, e não necessariamente a busca.**

1. Listas encadeadas e listas ligadas são dois nomes para a mesma coisa (na verdade, são traduções diferentes do mesmo termo em inglês: *linked lists*).
2. O Java já tem uma implementação própria disso: o ***LinkedList***. Contudo, como o nosso foco é saber como essas estruturas funcionam, vimos uma implementação do zero deste tipo de lista.
3. Existem dois tipos de listas encadeadas:
 - Simples: possuem uma conexão (ponteiro) para o próximo elemento.
 - Duplas: possuem uma conexão para o próximo elemento, e uma outra conexão para o elemento anterior.
4. Listas duplamente encadeadas são mais rápidas para inserir e remover elementos do que listas encadeadas simples, mas usam mais espaço para isso.

| Conclusão

Vimos nesta unidade três estruturas de dados básicas: vetores, matrizes e listas ligadas. Como já comentamos algumas vezes, tais estruturas já existem “de fábrica” no Java, com *ArrayList* e *LinkedList*. Contudo, nem todas as linguagens e *frameworks* possuem essas estruturas. Portanto, é importante sabermos como elas funcionam e as suas vantagens e desvantagens.

No mercado de trabalho, você pode se deparar com alguma situação na qual estruturas assim possam ser necessárias para resolver um problema, mas ao mesmo tempo não teria a possibilidade técnica de usar algo pronto como um *ArrayList* ou *LinkedList*: é por este motivo que apresentamos estas estruturas para você.

| Referências

ASCENCIO, A; ARAÚJO, G. **Estrutura de dados**: Algoritmos, análise da complexidade e implementações em Java e C/C++. São Paulo, SP: Pearson, 2010.

CELES, W.; CERQUEIRA, R.; RANGEL, J. **Introdução à estrutura de dados**. Minas Gerais, BH: Book, 2016.

TENENBAUM, A. M. **Estrutura de dados usando C**. São Paulo, SP: Makron Books, 1995.



© PUCPR - Todos os direitos reservados.